

Module Coursework Coversheet

Blind Grading Number: 8864G
Module Code: 4F13
Module Name: Probabilistic Machine Learning
Coursework Number or Name: 2: Probabilistic Ranking
Word or Page Count (if limit prescribed): 5 pages (cover sheet excluded)

Declaration

☒ <i>tick to confirm</i>	I hereby declare that this piece of work is my own unaided effort and adheres to the Department of Engineering's guidelines on plagiarism.
--	--

This piece of work is to be marked according to the following grading scheme:

Distinction		Pass				Fail (C+ - marginal fail)		
Outstanding	A+	A	A-	B+	B	C+	C	Unsatisfactory
90-100	80-89	75-79	70-74	65-69	60-64	55-59	50-54	0-49

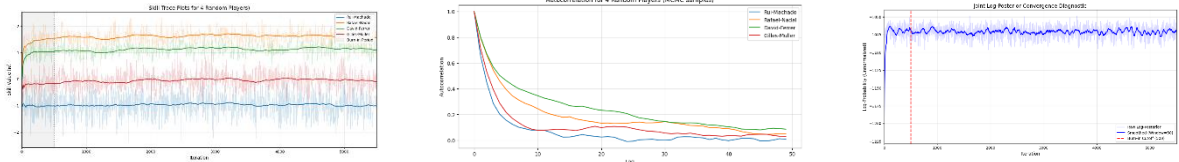
Late Penalty: 10% of mark for each day, or part day, late (Sunday excluded).

Coursework 2: Probabilistic Ranking

Question A

Command 1 Metropolis-Hastings MCMC algorithm

```
w_prop = w[i] + np.random.normal(0, proposal_sigma)
lp2 = norm.logpdf(w_prop) + np.sum(norm.logcdf(outcome * (w_prop - w[j])))
if np.log(np.random.rand()) < lp2-lp1:
    w[i] = w_prop
```



(a) Trace plot for random players (b) Autocorrelation for random players (c) log-posterior & burn-in

Figure 1 MCMC results for 5500 iterations

The sampler was run for 5500 iterations, achieving an acceptance rate of 31.84%. This falls within the optimal range for random walk Metropolis, confirming our choice of σ_{prop} was appropriate.

Figure 1(a) shows all chains initialize at the prior mean. At the early stage, we see a rapid drift as the skills move towards their high-probability regions. **Figure 1(c)** tracks the joint log-posterior, both of them indicate that the chains stabilize and begin fluctuating after approximately 300-400 iterations. To be conservative and remove initialization bias, we discard the first 500 samples as **burn-in**.

Figure 1(b) displays the autocorrelation functions for the same four players. We observe two distinct behaviors -- fast and slow mixing. For players with fewer games, the autocorrelation drops to zero very quickly. Their posterior is dominated by the prior, making it easy to explore. For top players, like the green curves, autocorrelation remains high. This is because their skills are tightly constrained by a large number of game outcomes, creating a narrow or correlated posterior landscape that the random walk explores slowly. To determine the necessary **number of iterations**, we analyze the autocorrelation time and the desired precision of our estimates. From **Figure 1(b)**, the autocorrelation for top players remains significant up to lag $\tau = 50$. Using the relationship between Monte Carlo Standard Error (MCSE) and chain length N , $MCSE \approx \frac{\sigma_{post}}{\sqrt{N/\tau}}$, assuming the posterior standard deviation $\sigma_{post} = 1$, and aiming for a target standard error of 0.1 (which corresponds to an Effective Sample Size of 100), we can solve for $N \approx \tau \times \left(\frac{\sigma}{target\ error}\right)^2 = 5000$. Adding the burn-in period of 500 iterations, the total recommended run length is **5500 iterations**. This ensures that despite the slow mixing, we obtain sufficient independent samples to characterize the posterior distribution reliably.

The computational complexity of the sampler is $\mathcal{O}(T \times G)$, where T is the number of iterations and G is the total number of games. In one full sweep of M players, the likelihood calculation visits every game exactly twice, once for each player. Thus, the cost per iteration scales linearly with the dataset size, not the number of players squared, making it efficient for sparse graphs like this dataset.

Question B

The nature of convergence is fundamentally different for the two methods. **Monte Carlo Sampling (MCMC)** exhibits stochastic convergence where the algorithm generates a Markov chain whose stationary distribution is the true posterior distribution $p(w|\mathcal{D})$. The object we converge to is this distribution itself. Thus, even after the chain has passed the burn-in phase and converged, the samples continue to fluctuate randomly as they explore the high-probability density regions.

In contrast, **Expectation Propagation (EP)** exhibits deterministic convergence. It approximates the posterior with a tractable Gaussian $q(w) = \prod_i \mathcal{N}(w_i; \mu_i, \sigma_i^2)$ by iteratively minimizing the local Kullback-Leibler divergence $KL(p||q)$ via moment matching. The object we converge to is a fixed set of parameters (means μ and variances Σ) that define this optimal approximation. Once converged, the values cease to change.

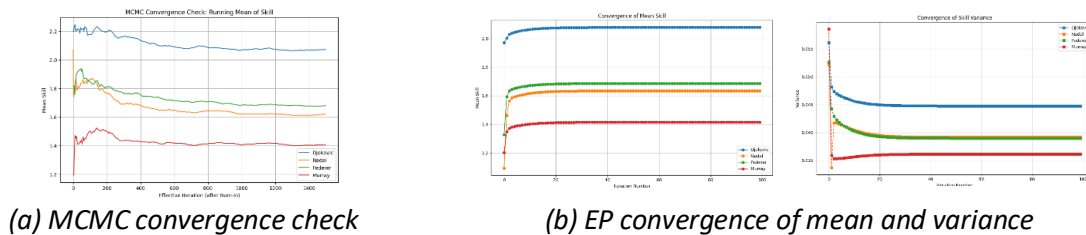


Figure 2 MCMC and EP convergence analysis

We judge **the convergence of MCMC** diagnostically by inspecting **Figure 2(a)** for stability, ensuring the chain oscillates around a stable mean without drift and calculating autocorrelation to estimate the mixing time required to generate independent samples.

For EP, we analyzed the convergence behavior using the parameter trajectories and the update magnitude. First, as shown in **Figure 2(b)**, both the mean skills and variances for the top 4 players change rapidly in the first 1-3 iterations and then flatten out into horizontal lines.

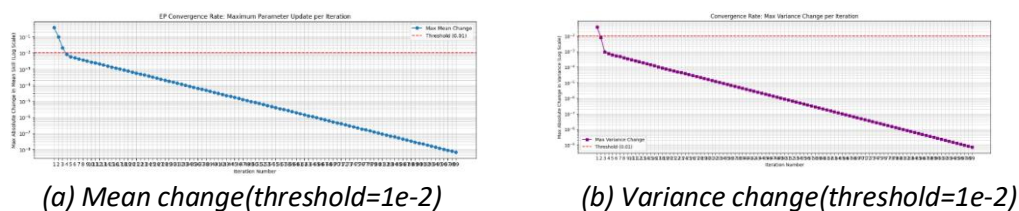


Figure 3 EP Max Absolute Change in Mean (Log Scale)

To rigorously judge convergence, we monitored the maximum parameter update Δ_t across all players. **Figure 3** shows that Δ_t drops exponentially. Using a threshold of 10^{-2} , the algorithm converged in about 4 iterations.

In conclusion, **MCMC requires thousands of iterations** to reduce Monte Carlo error and account for slow mixing, whereas **EP converges in just 5 to 6 iterations**. This stark difference arises because the TrueSkill posterior is unimodal and log-concave. EP effectively solves a system of coupled equations to match the mode and curvature of the distribution. The coordinate-ascent-like updates of EP propagate information globally across the factor graph very efficiently, allowing it to snap to the

optimal Gaussian approximation almost instantly compared to the random walk exploration of MCMC.

Question C

Upon convergence of the Expectation Propagation (EP) algorithm, we obtained the approximate Gaussian marginals for each player's skill $q(w_i) = \mathcal{N}(\mu_i, \sigma_i^2)$.

Probability of Higher Skill reflects the epistemic uncertainty regarding which player possesses a higher latent skill. It is computed as the probability that the difference of two Gaussian variables is positive, where the variance of the difference is the sum of individual skill variances.

$$P(s_A > s_B) = \Phi\left(\frac{\mu_A - \mu_B}{\sqrt{\sigma_A^2 + \sigma_B^2}}\right)$$

Probability of Winning predicts the outcome of a specific match. The TrueSkill generative model assumes a match outcome is determined by performance $p \sim \mathcal{N}(s, \beta^2)$ where $\beta^2 = 1$ represents aleatoric performance noise.

$$P(y_{AB}) = \Phi\left(\frac{\mu_A - \mu_B}{\sqrt{\sigma_A^2 + \sigma_B^2 + 1}}\right)$$

Command 2 Codes for 2 probability computation

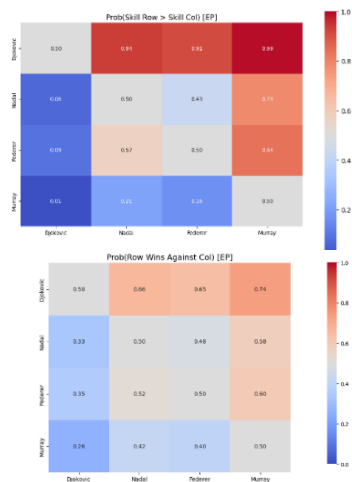
```
prob_skill = norm.cdf((mus[i] - mus[j]) / np.sqrt(vars[i] + vars[j]))
prob_win = norm.cdf((mus[i] - mus[j]) / np.sqrt(vars[i] + vars[j] + 1.0))
```

Table 1 $P(\text{Skill Row} > \text{Skill Col})$ using EP

	Djokovic	Nadal	Federer	Murray
Djokovic	0.500	0.937	0.912	0.990
Nadal	0.063	0.500	0.426	0.788
Federer	0.088	0.574	0.500	0.839
Murray	0.010	0.212	0.161	0.500

Table 2 $P(\text{Row wins Col})$ using EP

	Djokovic	Nadal	Federer	Murray
Djokovic	0.500	0.665	0.646	0.737
Nadal	0.334	0.500	0.480	0.584
Federer	0.353	0.520	0.500	0.603
Murray	0.261	0.416	0.397	0.500



Comparing Table 1 and Table 2, the **Winning Probabilities** are consistently closer to 0.5 (more conservative) than the **Skill Probabilities**. For instance, while the model is highly confident Djokovic has a higher skill than Nadal ($P \approx 0.937$), the predicted win probability is lower ($P \approx 0.665$).

As shown in **Figure 4**, the Skill Difference distribution (solid line) has a smaller variance, resulting in a sharper peak. The Performance Difference distribution (dashed line) incorporates the aleatoric noise $\beta^2 = 1$, resulting in a much wider and flatter distribution. Structurally, the term β^2 in the

denominator of the Winning Probability formula reduces the Z-score argument passed to the cumulative density function $\Phi(\cdot)$. Consequently, the probability mass is pulled towards the center (0.5), reflecting the reality that even a significantly more skilled player is not guaranteed to win every single match due to inherent variance in performance.

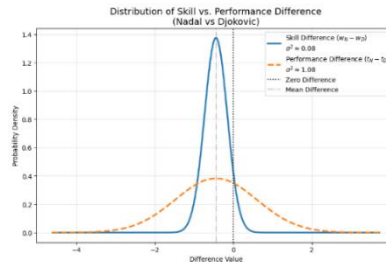


Figure 4 Sample distribution of skill & performance

Question D

We evaluated three methods to estimate the probability that Nadal has a higher skill than Federer using the MCMC samples. **Marginal Gaussian** approximates the posterior as independent Gaussians $q(w_N)q(w_F)$, calculating probability via the CDF of the difference, ignoring covariance. **Joint Gaussian** approximates the joint posterior as a multivariate Gaussian. It accounts for correlation via the variance of the difference $\sigma_{diff}^2 = \sigma_N^2 + \sigma_F^2 - 2Cov(w_N, w_F)$. **Empirical** is a non-parametric Monte Carlo integration that counts the proportion of samples where $w_N > w_F$.

Command 3 core codes to compare 3 methods

```
prob_marginal = norm.cdf((mean_N - mean_F) / np.sqrt(var_N + var_F))
prob_joint = norm.cdf(np.mean(samples_N - samples_F) / np.std(samples_N - samples_F))
prob_empirical = np.mean(samples_N > samples_F)
```

Table 3 $P(\text{Skill Nadal} > \text{Skill Federer})$ using 3 methods

	P(Skill Nadal > Skill Federer)
Marginal Approx (Ignored Corr)	0.4224
Joint Approx (Captured Corr)	0.4117
Empirical (Samples)	0.4207

Joint Gaussian Approximation is the best method for this application and in most cases. Unlike **Marginal Gaussian Approximation**, it fully accounts for the correlations between player skills induced by match outcomes. Ignoring these correlations as Marginal Gaussian does will lead to incorrect variance estimates and biased probabilities. While **Empirical method** is asymptotically exact, it is sensitive to Monte Carlo sampling noise, particularly in the tails of the distribution where data is sparse, so it needs sufficient data to converge. Joint Gaussian Approximation imposes a structural Gaussian assumption on the posterior. Since the TrueSkill posterior is unimodal and log-concave, this assumption acts as a regularizer, smoothing out sampling noise and providing a more robust density estimate than the raw histogram of the empirical method. **But** as there are some non-gaussian data, Empirical one sometimes performs better than Joint Gaussian one.

Joint Gaussian Approximation offers the optimal trade-off: it incorporates the necessary dependency structure (unlike Marginal Gaussian Approximation) while providing a smoother, analytically tractable estimate that is robust to finite sample size (unlike Empirical method).

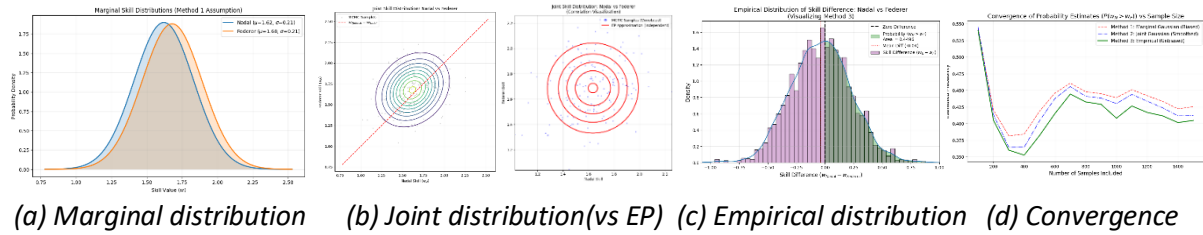


Figure 5 Methods distribution visualization

We generated the **Table 4** using the Joint Gaussian method and compared it with the EP results from question C. The results from MCMC align very closely with the EP estimates. This reinforces the validity of the Gaussian approximation for this domain. However, the small remaining differences highlight a key theoretical distinction: while EP accurately captures the marginal means and variances, its fully factorized approximation $q(w) = \prod q(w_i)$ explicitly assumes independence between players. Consequently, EP misses the full covariance structure that Joint Gaussian successfully extracts from the MCMC samples, though this omission appears to have a minor impact on the final ranking probabilities for the top players.

Table 4 $P(\text{Skill Row} > \text{Skill Col})$ using MCMC (Joint Gaussian) & heatmap visualization(vs. EP)

	Djokovic	Nadal	Federer	Murray
Djokovic	0.500	0.948	0.912	0.991
Nadal	0.052	0.500	0.412	0.800
Federer	0.088	0.588	0.500	0.833
Murray	0.009	0.200	0.167	0.500

Question E

In **Figure 6**, the rankings produced by **MCMC** and **EP** are virtually identical, confirming that the deterministic EP algorithm successfully approximates the first-order statistics of the true posterior skill distribution required for ranking. In stark contrast, the **Empirical** ranking based on raw win rates differs significantly, producing a shuffled order that fails to reflect the true hierarchy of skill.

The **Empirical** method is computationally trivial but fundamentally flawed: it ignores opponent strength, treating a win against a novice the same as one against a top seed, and lacks a generative model to predict outcomes for players who have never met. Crucially, it fails to quantify uncertainty, making it unreliable for players with sparse data. **MCMC** provides the asymptotically exact "gold standard" posterior and captures full uncertainty, but it is computationally expensive and requires careful monitoring of stochastic convergence. **EP** emerges as the optimal choice for practical ranking applications. It offers efficient deterministic convergence while maintaining ranking accuracy comparable to the expensive MCMC method. Although Task (d) revealed that EP ignores joint correlations, this limitation proves negligible for the specific task of ranking. By explicitly quantifying uncertainty and accounting for opponent strength with minimal computational cost, EP is superior to the context-blind Empirical method and the slower MCMC sampler for real-time systems.

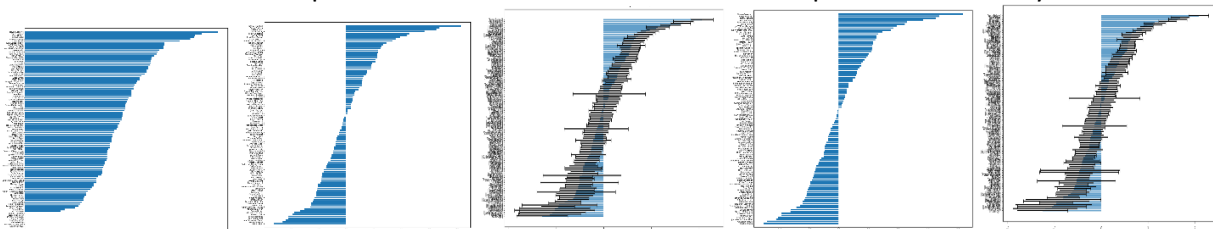


Figure 6 Player order with different methods (left to right: Empirical, MCMC, EP)